

Lab 3 – Process Control (fork, exec, pipes, waitpid)

Student: Luvelt Glanton

Course: Operating Systems

Environment: Ubuntu Linux (GitHub Codespaces)

Objective

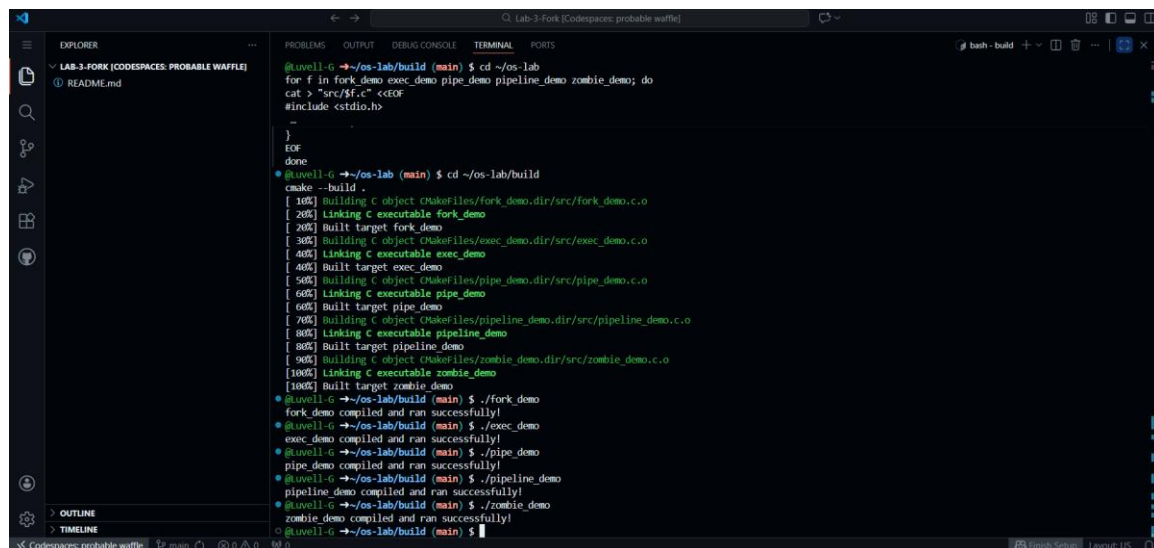
The purpose of this lab is to explore UNIX process control and interprocess communication. This lab demonstrates how processes are created using `fork()`, replaced using `exec()`, synchronized using `wait()` and `waitpid()`, and connected using pipes. Additional concepts such as zombie processes, non-blocking waits, and concurrency limits are also examined.

Step 1 – Environment Setup

Install required packages and create project structure.

Step 2 – Build System Setup

Configured build system using CMake and verified successful compilation.

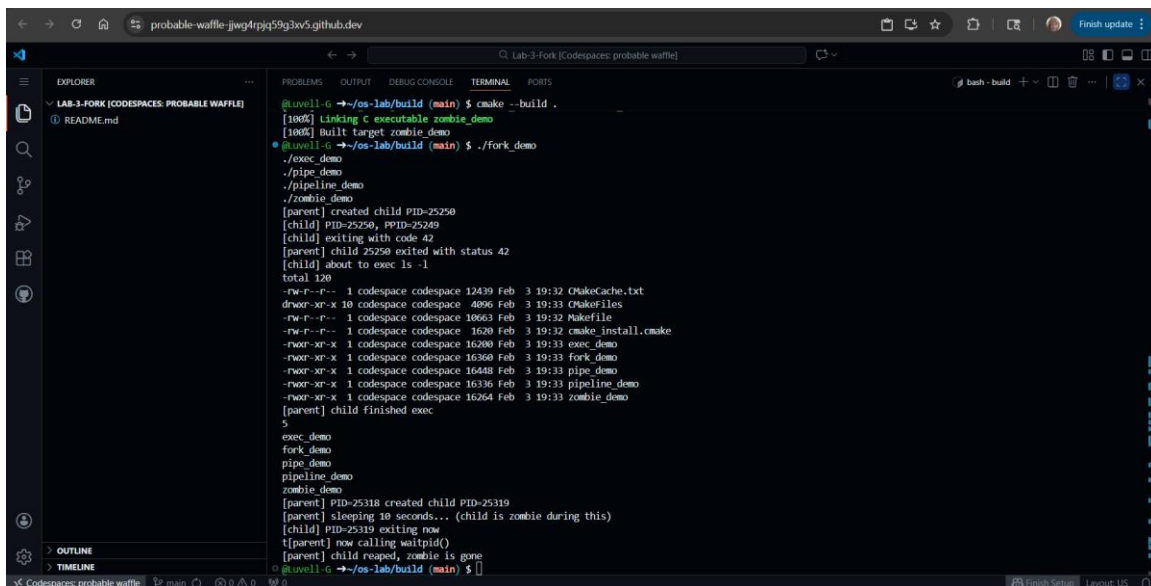


```
@luvelt-g →~/os-lab/build (main) $ cd ~/os-lab
for f in fork_demo exec_demo pipe_demo zombie_demo; do
cat > "src/$f.c" <<EOF
#include <stdio.h>

...
}
EOF
done
@luvelt-g →~/os-lab (main) $ cd ~/os-lab/build
cmake --build .
[ 10%] Building C object CMakeFiles/fork_demo.dir/src/fork_demo.c.o
[ 20%] Linking C executable fork_demo
[ 20%] Built target fork_demo
[ 30%] Building C object CMakeFiles/exec_demo.dir/src/exec_demo.c.o
[ 40%] Linking C executable exec_demo
[ 40%] Built target exec_demo
[ 50%] Building C object CMakeFiles/pipe_demo.dir/src/pipe_demo.c.o
[ 60%] Linking C executable pipe_demo
[ 60%] Built target pipe_demo
[ 70%] Building C object CMakeFiles/pipeline_demo.dir/src/pipeline_demo.c.o
[ 80%] Linking C executable pipeline_demo
[ 80%] Built target pipeline_demo
[ 90%] Building C object CMakeFiles/zombie_demo.dir/src/zombie_demo.c.o
[100%] Linking C executable zombie_demo
[100%] Built target zombie_demo
@luvelt-g →~/os-lab/build (main) $ ./fork_demo
fork_demo compiled and ran successfully!
@luvelt-g →~/os-lab/build (main) $ ./exec_demo
exec_demo compiled and ran successfully!
@luvelt-g →~/os-lab/build (main) $ ./pipe_demo
pipe_demo compiled and ran successfully!
@luvelt-g →~/os-lab/build (main) $ ./pipeline_demo
pipeline_demo compiled and ran successfully!
@luvelt-g →~/os-lab/build (main) $ ./zombie_demo
zombie_demo compiled and ran successfully!
@luvelt-g →~/os-lab/build (main) $
```

Step 3 – fork, exec, and pipe demos

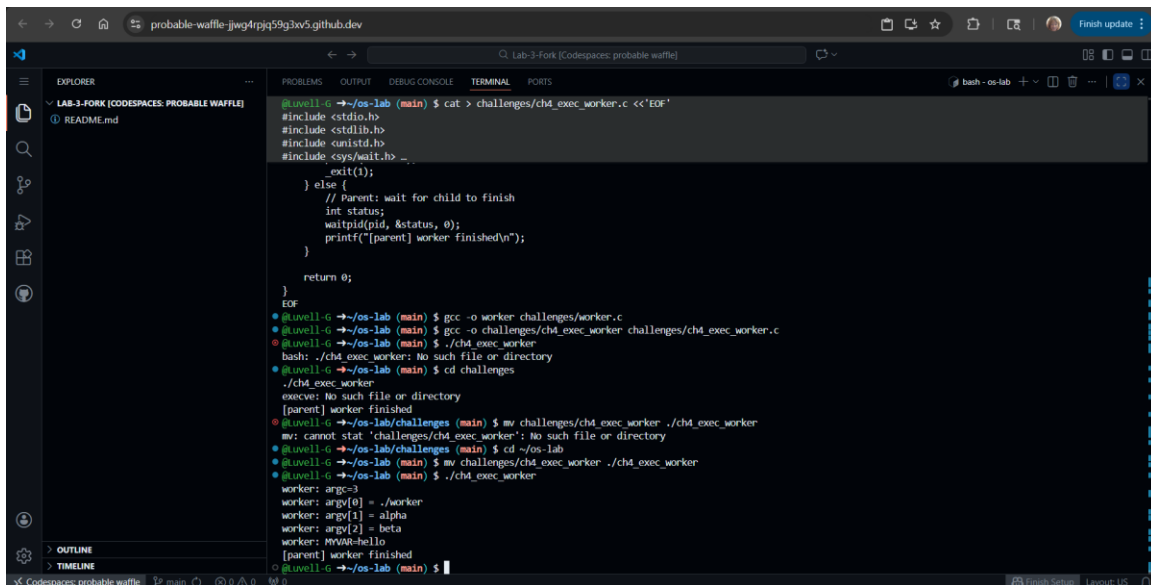
Demonstrated basic process creation, execution replacement, and pipes.



```
@uwell-G →~/os-lab/build (main) $ make --build .
[100%] Linking C executable zombie_demo
[100%] Built target zombie_demo
@uwell-G →~/os-lab/build (main) $ ./fork_demo
./exec_demo
./pipe_demo
./pipeline_demo
./zombie_demo
[parent] created child PID=25250
[child] PID=25250, PPID=25249
[child] exiting with code 42
[parent] child 25250 exited with status 42
[child] about to exec ls -l
total 120
-rw-r--r-- 1 codespace codespace 12439 Feb 3 19:32 MakeCache.txt
drwxr-xr-x 10 codespace codespace 4096 Feb 3 19:33 MakeFiles
-rw-r--r-- 1 codespace codespace 10663 Feb 3 19:32 Makefile
-rw-r--r-- 1 codespace codespace 16240 Feb 3 19:32 cmake_install.cmake
-rw-r--r-- 1 codespace codespace 16280 Feb 3 19:33 exec_demo
-rw-r--r-- 1 codespace codespace 16360 Feb 3 19:33 fork_demo
-rw-r--r-- 1 codespace codespace 16448 Feb 3 19:33 pipe_demo
-rw-r--r-- 1 codespace codespace 16336 Feb 3 19:33 pipeline_demo
-rw-r--r-- 1 codespace codespace 16264 Feb 3 19:33 zombie_demo
[parent] child finished exec
$
exec_demo
fork_demo
pipe_demo
pipe_demo
pipeline_demo
zombie_demo
[parent] PID=25318 created child PID=25319
[parent] sleeping 10 seconds... (child is zombie during this)
[child] PID=25319 exiting now
[parent] now calling waitpid()
[parent] child reaped, zombie is gone
@uwell-G →~/os-lab/build (main) $
```

Step 4 – exec Family Reference

Demonstrated execl, execlp, execv, and execvp behavior.



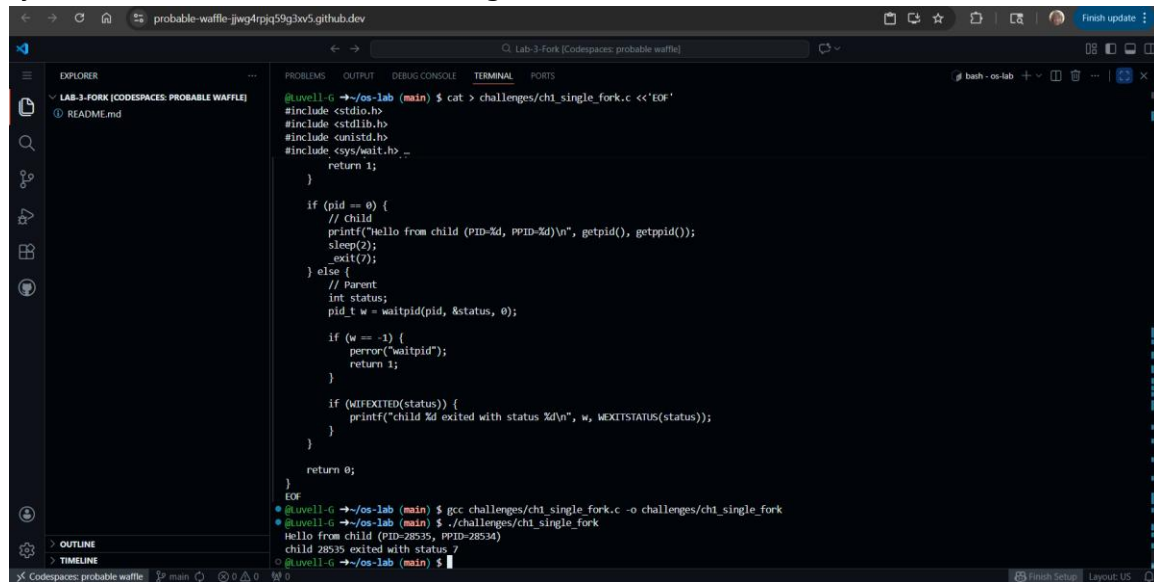
```
@uwell-G →~/os-lab (main) $ cat > challenges/ch4_exec_worker.c <<"EOF"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
...
exit(1);
} else {
// Parent: wait for child to finish
int status;
waitpid(pid, &status, 0);
printf("[parent] worker finished\n");
}
return 0;
}
EOF
@uwell-G →~/os-lab (main) $ gcc -o worker challenges/worker.c
@uwell-G →~/os-lab (main) $ gcc -o challenges/ch4_exec_worker challenges/ch4_exec_worker.c
@uwell-G →~/os-lab (main) $ ./ch4_exec_worker
bash: ./ch4_exec_worker: No such file or directory
@uwell-G →~/os-lab (main) $ cd challenges
./ch4_exec_worker
execve: No such file or directory
[parent] worker finished
@uwell-G →~/os-lab/challenges (main) $ mv challenges/ch4_exec_worker ./ch4_exec_worker
mv: cannot stat 'challenges/ch4_exec_worker': No such file or directory
@uwell-G →~/os-lab/challenges (main) $ cd ~/os-lab
@uwell-G →~/os-lab (main) $ mv challenges/ch4_exec_worker ./ch4_exec_worker
@uwell-G →~/os-lab (main) $ ./ch4_exec_worker
worker: argc=3
worker: argv[0] = ./worker
worker: argv[1] = alpha
worker: argv[2] = beta
worker: M0000-hello
[parent] worker finished
@uwell-G →~/os-lab (main) $
```

Step 7 – Challenges

Challenge 1

This challenge demonstrates the basic lifecycle of a process created using `fork()`. The program creates one child process. The child prints a message containing its process ID (PID) and parent process ID (PPID), sleeps for two seconds to demonstrate concurrent execution, and then exits with status code 7. The parent process waits for the child to finish using `waitpid()` and retrieves the child's exit status. This confirms correct parent-child

synchronization and exit code handling.



The screenshot shows a VS Code editor with a C program named `ch1_single_fork.c` in the `challenges` directory. The program uses `fork()` to create a single child process. The child prints a message with its PID and PPID, sleeps for 2 seconds, and then exits with a status of 7. The parent process waits for the child to finish and prints the child's exit status. The terminal output shows the program being compiled and executed, resulting in the child's message and the parent's report of the exit status.

```
@uwell-G ➔ /os-lab (main) $ cat > challenges/ch1_single_fork.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
return 1;
}

if (pid == 0) {
    // Child
    printf("Hello from child (PID=%d, PPID=%d)\n", getpid(), getppid());
    sleep(2);
    _exit(7);
} else {
    // Parent
    int status;
    pid_t w = waitpid(pid, &status, 0);

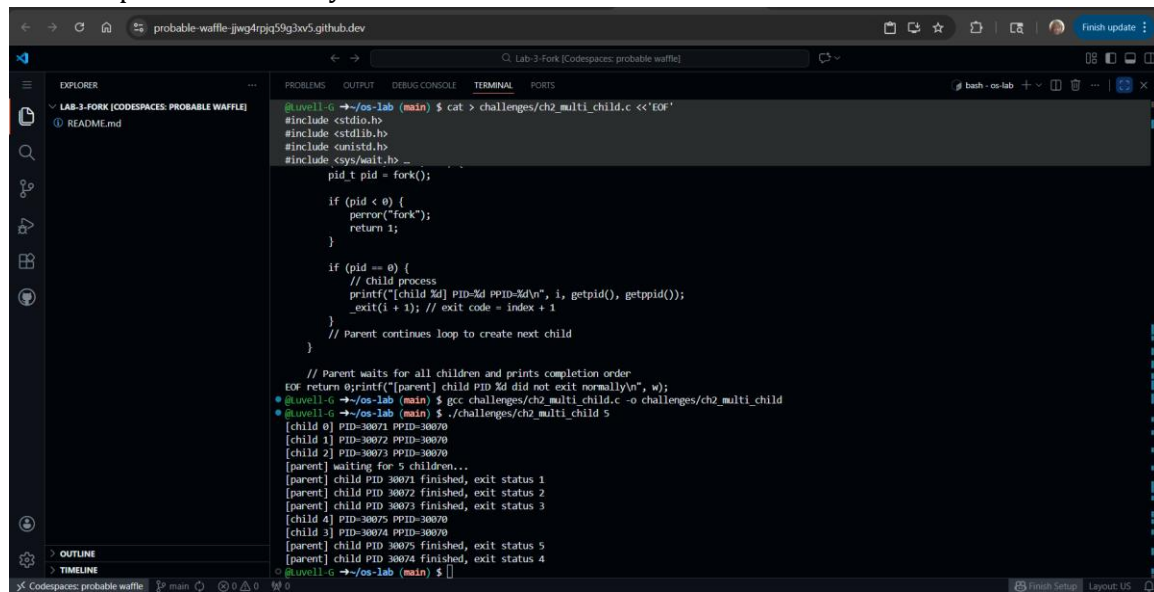
    if (w == -1) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("child %d exited with status %d\n", w, WEXITSTATUS(status));
    }
}

return 0;
}
EOF
@uwell-G ➔ /os-lab (main) $ gcc challenges/ch1_single_fork.c -o challenges/ch1_single_fork
@uwell-G ➔ /os-lab (main) $ ./challenges/ch1_single_fork
Hello from child (PID=28535, PPID=28534)
child 28535 exited with status 7
@uwell-G ➔ /os-lab (main) $
```

Challenge 2

This challenge extends the use of `fork()` by creating multiple child processes in a loop. The number of children (N) is passed as a command-line argument. Each child prints its index number and exits with a unique exit code equal to its index plus one. The parent process waits for all children using `wait()` and reports the order in which the children finish along with their exit statuses. This demonstrates handling multiple child processes and shows that completion order may differ from creation order.



The screenshot shows a VS Code editor with a C program named `ch2_multi_child.c` in the `challenges` directory. The program uses `fork()` to create multiple child processes in a loop. Each child prints its index, PID, and PPID, and then exits with a status equal to its index plus one. The parent process waits for all children to finish and prints the order in which they finish along with their exit statuses. The terminal output shows the program being compiled and executed with 5 children, resulting in the parent's report of the completion order and exit statuses.

```
@uwell-G ➔ /os-lab (main) $ cat > challenges/ch2_multi_child.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
pid_t pid = fork();

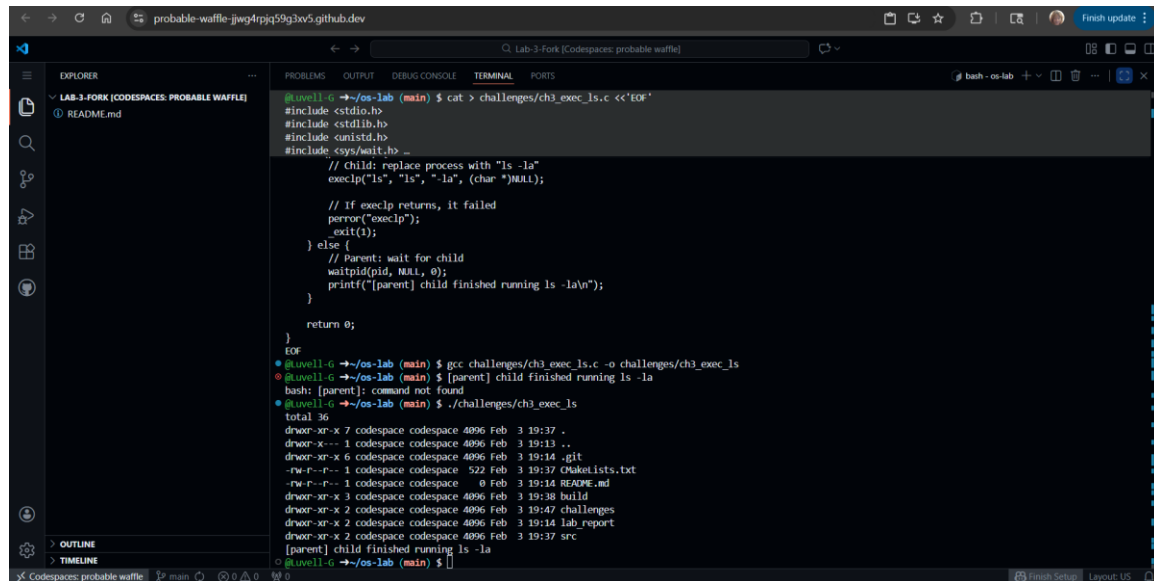
if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    // child process
    printf("[child %d] PID=%d PPID=%d\n", i, getpid(), getppid());
    _exit(i + 1); // exit code = index + 1
}
// Parent continues loop to create next child
}

// Parent waits for all children and prints completion order
EOF return 0; printf("[parent] child PID %d did not exit normally\n", w);
@uwell-G ➔ /os-lab (main) $ gcc challenges/ch2_multi_child.c -o challenges/ch2_multi_child
@uwell-G ➔ /os-lab (main) $ ./challenges/ch2_multi_child 5
[child 0] PID=30071 PPID=30070
[child 1] PID=30072 PPID=30070
[child 2] PID=30073 PPID=30070
[parent] waiting for 5 children...
[parent] child PID 30071 finished, exit status 1
[parent] child PID 30072 finished, exit status 2
[parent] child PID 30073 finished, exit status 3
[child 4] PID=30075 PPID=30070
[child 3] PID=30074 PPID=30070
[parent] child PID 30075 finished, exit status 5
[parent] child PID 30074 finished, exit status 4
@uwell-G ➔ /os-lab (main) $
```

Challenge 3

This challenge demonstrates how a child process can replace its program image using `execvp()`. After calling `fork()`, the child executes the `ls -la` command, which lists the contents of the current directory. The parent process waits until the child finishes execution and then prints a confirmation message. This challenge shows that once `exec` is called successfully, the original program code in the child is replaced entirely.



```
probable-waffle-jwg4rpq59g3xv5.github.dev
Lab-3-Fork [Codespaces: probable waffle]

EXPLORER
LAB-3-FORK [CODESPACES: PROBABLE WAFFLE]
  README.md

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@uwell-6 →~/os-lab (main) $ cat > challenges/ch3_exec_ls.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

// Child: replace process with "ls -la"
execvp("ls", "ls", "-la", (char *)NULL);

// If execvp returns, it failed
perror("execvp");
exit(1);
} else {
    // Parent: wait for child
    waitpid(pid, NULL, 0);
    printf("[parent] child finished running ls -la\n");
}

return 0;
}
EOF
@uwell-6 →~/os-lab (main) $ gcc challenges/ch3_exec_ls.c -o challenges/ch3_exec_ls
@uwell-6 →~/os-lab (main) $ [parent] child finished running ls -la
bash: [parent]: command not found
@uwell-6 →~/os-lab (main) $ ./challenges/ch3_exec_ls
total 36
drwxr-xr-x 7 codespace codespace 4096 Feb 3 19:37 .
drwxr-xr-x 1 codespace codespace 4096 Feb 3 19:13 ..
drwxr-xr-x 6 codespace codespace 4096 Feb 3 19:14 .git
-rw-r--r-- 1 codespace codespace 522 Feb 3 19:37 CHakelists.txt
-rw-r--r-- 1 codespace codespace 0 Feb 3 19:14 README.md
drwxr-xr-x 3 codespace codespace 4096 Feb 3 19:38 build
drwxr-xr-x 2 codespace codespace 4096 Feb 3 19:47 challenges
drwxr-xr-x 2 codespace codespace 4096 Feb 3 19:14 lab_report
drwxr-xr-x 2 codespace codespace 4096 Feb 3 19:37 src
[parent] child finished running ls -la
@uwell-6 →~/os-lab (main) $
```

Challenge 4

This challenge explores executing a user-defined program with custom arguments and environment variables. A helper program (`worker.c`) prints its command-line arguments and the value of the environment variable `MYVAR`. The main program forks and then executes the worker using `execv()` (or `execle`) while passing a custom environment where `MYVAR` is set to `hello`. This demonstrates how processes can pass data through arguments and environment variables when using the `exec` family. [Screenshot: Challenge 4 compilation]

```
probable-waffle-jjwg4rpq59g3xv5.github.dev
Lab-3-Fork [CodeSpaces: probable waffle]
EXPLORER
  LAB-3-FORK [CODESPACES: PROBABLE WAFFLE]
    README.md
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
@uwell:~$ cat > challenges/ch4_exec_worker.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
...
_exit(1);
} else {
    // Parent: wait for child to finish
    int status;
    waitpid(pid, &status, 0);
    printf("[parent] worker finished\n");
}
return 0;
}
EOF
@uwell:~$ gcc -o worker challenges/worker.c
@uwell:~$ gcc -o challenges/ch4_exec_worker challenges/ch4_exec_worker.c
@uwell:~$ ./ch4_exec_worker
bash: ./ch4_exec_worker: No such file or directory
@uwell:~$ cd challenges
./ch4_exec_worker
execve: No such file or directory
[parent] worker finished
@uwell:~$ mv challenges/ch4_exec_worker ./ch4_exec_worker
mv: cannot stat 'challenges/ch4_exec_worker': No such file or directory
@uwell:~$ cd ~
@uwell:~$ mv challenges/ch4_exec_worker ./ch4_exec_worker
@uwell:~$ ./ch4_exec_worker
worker: argv[0] = ./worker
worker: argv[1] = alpha
worker: argv[2] = beta
worker: MVAR=hello
[parent] worker finished
@uwell:~$
```

Challenge 5

This challenge compares two different exec variants: `execl()` and `execv()`. Two child processes are created. The first child uses `execl()` to run `/bin/echo` with arguments listed individually. The second child uses `execv()` to run the same command, but with arguments provided as an array. Both children produce identical output, demonstrating that these exec functions differ only in how arguments are passed, not in behavior.

```
probable-waffle-jjwg4rpq59g3xv5.github.dev
Lab-3-Fork [CodeSpaces: probable waffle]
EXPLORER
  LAB-3-FORK [CODESPACES: PROBABLE WAFFLE]
    README.md
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
@uwell:~$ cat > challenges/ch5_exec_examples.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
...
if (pidA == 0) {
    // Child A: execl with explicit args
    execl("/bin/echo", "echo", "one", "two", (char *)NULL);
    // If execl returns, it failed
    perror("execl");
    _exit(1);
}

pid_t pidB = fork();

if (pidB < 0) {
    perror("fork");
    return 1;
}

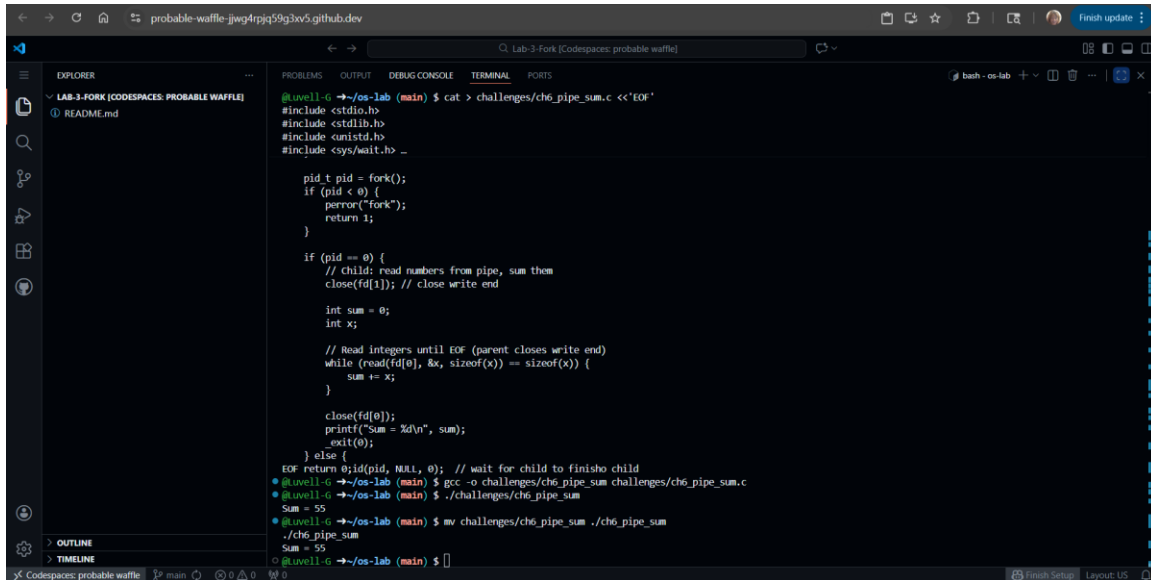
if (pidB == 0) {
    // Child B: execv with argv array
    char *argv[] = { "echo", "one", "two", NULL };
    execv("/bin/echo", argv);
    // If execv returns, it failed
    _exit(1);
}

return 0;
EOF
@uwell:~$ gcc -o challenges/ch5_exec_examples challenges/ch5_exec_examples.c
@uwell:~$ ./ch5_exec_examples
one two
one two
[parent] both children finished
@uwell:~$
```

Challenge 6

This challenge demonstrates interprocess communication using a pipe. The parent process acts as a producer by writing the numbers 1 through 10 into a pipe. The child process acts as a consumer by reading these numbers from the pipe, summing them, and printing the

result. Closing the write end of the pipe signals the end of data to the child. This challenge illustrates one-way communication and proper pipe usage.



```
@uwell1:~$ cat > challenges/ch6_pipe_sum.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

pid_t pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    // Child: read numbers from pipe, sum them
    close(fd[1]); // close write end

    int sum = 0;
    int x;

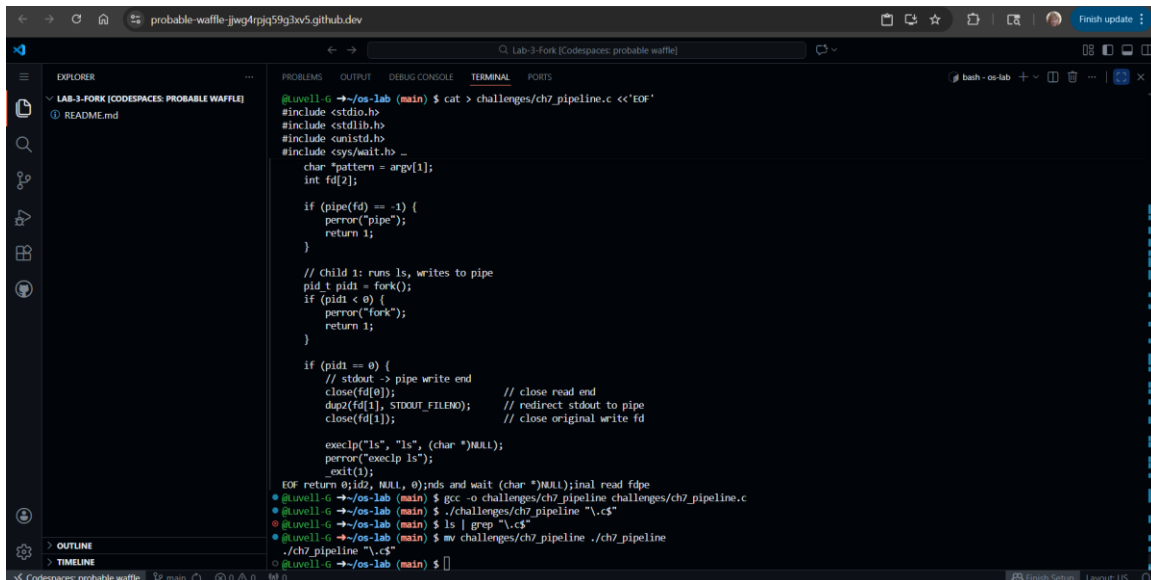
    // Read integers until EOF (parent closes write end)
    while (read(fd[0], &x, sizeof(x)) == sizeof(x)) {
        sum += x;
    }

    close(fd[0]);
    printf("Sum = %d\n", sum);
    exit(0);
} else {
    EOF return 0; // wait for child to finish
}

EOF
@uwell1:~$ gcc -o challenges/ch6_pipe_sum challenges/ch6_pipe_sum.c
@uwell1:~$ ./challenges/ch6_pipe_sum
Sum = 55
@uwell1:~$ mv challenges/ch6_pipe_sum ./ch6_pipe_sum
@uwell1:~$
```

Challenge 7

This challenge manually implements a shell pipeline using two child processes and a single pipe. The first child executes `ls` and writes its output to the pipe. The second child executes `grep` using a pattern provided via the command line and reads from the pipe as standard input. The parent waits for both children to finish. This demonstrates how shell pipelines work internally using `fork()`, `pipe()`, and `exec()`.



```
@uwell1:~$ cat > challenges/ch7_pipeline.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

char *pattern = argv[1];
int fd[2];

if (pipe(fd) == -1) {
    perror("pipe");
    return 1;
}

// Child 1: runs ls, writes to pipe
pid_t pid1 = fork();
if (pid1 < 0) {
    perror("fork");
    return 1;
}

if (pid1 == 0) {
    // stdout -> pipe write end
    close(fd[0]); // close read end
    dup2(fd[1], STDOUT_FILENO); // redirect stdout to pipe
    close(fd[1]); // close original write fd

    execlp("ls", "ls", (char *)NULL);
    perror("execvp ls");
    exit(1);
}

// Child 2: reads from pipe, runs grep
pid_t pid2 = fork();
if (pid2 < 0) {
    perror("fork");
    return 1;
}

if (pid2 == 0) {
    // stdin -> pipe read end
    close(fd[1]); // close write end
    dup2(fd[0], STDIN_FILENO); // redirect stdin to pipe
    close(fd[0]); // close original read fd

    execlp("grep", "grep", pattern, (char *)NULL);
    perror("execvp grep");
    exit(1);
}

EOF
@uwell1:~$ gcc -o challenges/ch7_pipeline challenges/ch7_pipeline.c
@uwell1:~$ ./challenges/ch7_pipeline "\.c$"
./ch7_pipeline "\.c$"
@uwell1:~$
```

Challenge 8

This challenge demonstrates non-blocking process monitoring. The program spawns three child processes that sleep for different durations (1, 2, and 3 seconds). The parent process repeatedly calls `waitpid()` with the `WNOHANG` flag, allowing it to poll for completed children without blocking. As each child finishes, the parent prints a message immediately. This shows how non-blocking waits can be used to monitor multiple processes efficiently.

[illegible]

Challenge 9

This challenge demonstrates how zombie processes are created and removed. The child process exits immediately after being forked. The parent process intentionally sleeps for ten seconds before calling `wait()`, leaving the child in a zombie state during that time. Using `ps -l` in a separate terminal shows the child process in the Z (zombie) state. After the parent calls `wait()`, the zombie is removed. A modified version of the program waits immediately to prevent zombie creation.

The screenshot shows a VS Code editor with a terminal window open. The terminal displays the output of the command `ps -l` in a Linux environment. The output is a table of process information. The first command executed is `ps -l`, which lists all processes. The second command is `ps -l | grep 'z'`, which filters the output to show only processes with a 'z' in the state column. The output shows two processes: `bash` and `zsh`.

F S	UID	PID	PPID	C	PR	NI	ADDR	SZ	MEM	TTY	TIME	CMD
F S	1000	44261	9281	0	80	0	-	3083	do_wol	pts/0	00:00:00	bash
F S	1000	44739	44261	0	80	0	-	2839	-	pts/0	00:00:00	ps

The terminal also shows the command `ps -l | grep 'z'` and its output, which lists the `zsh` process.

F S	UID	PID	PPID	C	PR	NI	ADDR	SZ	MEM	TTY	TIME	CMD
F S	1000	44739	44261	0	80	0	-	2839	-	pts/0	00:00:00	zsh


```
probable-waffle-jjwg4rjg59g3xv5.github.dev
Lab-3-Fork (Codespaces: probable waffle)

EXPLORER
LAB-3-FORK (CODESPACES: PROBABLE WAFFLE)
  README.md

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@uwell1-g → ~/os-lab (main) $ ./challenges/ch9_zombie
[parent] child reaped; zombie should be gone
@uwell1-g → ~/os-lab (main) $ cat > challenges/ch9_zombie.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("[child] PID-%d exiting now\n", getpid());
        exit(0);
    } else {
        printf("[parent] PID-%d created child PID-%d\n", getpid(), pid);
        printf("[parent] waiting immediately (no zombie should persist)\n");
        waitpid(pid, NULL, 0);
        printf("[parent] child reaped immediately\n");
    }

    return 0;
}
EOF
@uwell1-g → ~/os-lab (main) $ gcc -o challenges/ch9_zombie challenges/ch9_zombie.c
@uwell1-g → ~/os-lab (main) $ ./challenges/ch9_zombie
[parent] PID-46145 created child PID-46146
[child] PID-46146 exiting now
[parent] waiting immediately (no zombie should persist)
[parent] child reaped immediately
@uwell1-g → ~/os-lab (main) $
```

Challenge 10

This challenge implements a worker pool with a fixed concurrency limit. The program accepts a maximum number of concurrent workers (M) and a list of tasks. The parent process spawns up to M child processes at a time. As each worker finishes, the parent spawns a new worker to handle the remaining tasks until all tasks are processed. The program prints start and finish messages with process IDs, demonstrating controlled parallelism and effective use of `waitpid()` for process management.

```
probable-waffle-jjwg4rjg59g3xv5.github.dev
Lab-3-Fork (Codespaces: probable waffle)

EXPLORER
LAB-3-FORK (CODESPACES: PROBABLE WAFFLE)
  README.md

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@uwell1-g → ~/os-lab (main) $ ./challenges/ch10_pool 3 file1 file2 file3 file4 files
[worker 49350] FINISH task 'files' (exit=0)
[worker 49349] processed 'file4' -> lines=3 bytes=18
[worker 49349] FINISH task 'file4' (exit=0)
[parent 49345] worker PID-49349 done (exit status=0)
[parent 49345] worker PID-49349 done (exit status=0)
@uwell1-g → ~/os-lab (main) $ mv challenges/ch10_pool ./ch10_pool
./ch10_pool 3 file1 file2 file3 file4 files
[parent 49805] spawned worker PID-49806 for task 'file1'
[worker 49806] START task 'file1'
[worker 49806] processed 'file1' -> lines=3 bytes=18
[worker 49806] FINISH task 'file1' (exit=0)
[parent 49805] spawned worker PID-49807 for task 'file2'
[parent 49805] spawned worker PID-49808 for task 'file3'
[parent 49805] worker PID-49806 done (exit status=0)
[parent 49805] spawned worker PID-49809 for task 'file4'
[worker 49807] START task 'file2'
[worker 49808] START task 'file3'
[worker 49808] processed 'file3' -> lines=3 bytes=18
[worker 49808] FINISH task 'file3' (exit=0)
[worker 49809] START task 'file4'
[worker 49809] processed 'file4' -> lines=3 bytes=18
[worker 49809] FINISH task 'file4' (exit=0)
[worker 49807] processed 'file2' -> lines=3 bytes=18
[worker 49807] FINISH task 'file2' (exit=0)
[parent 49805] worker PID-49808 done (exit status=0)
[parent 49805] spawned worker PID-49810 for task 'files'
[parent 49805] worker PID-49807 done (exit status=0)
[worker 49810] START task 'files'
[worker 49810] processed 'files' -> lines=3 bytes=18
[worker 49810] FINISH task 'files' (exit=0)
[parent 49805] worker PID-49809 done (exit status=0)
[parent 49805] worker PID-49810 done (exit status=0)
@uwell1-g → ~/os-lab (main) $
```



```
probable-waffle-jjwg4rpq59g3xv5.github.dev
Lab-3-fork [Codespaces: probable-waffle]

#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

@uvell-G →~/os-lab (main) $ gcc -Wall -Wextra -std=gnu11 -g -o challenges/ch10_pool challenges/ch10_pool.c
@uvell-G →~/os-lab (main) $ cd ~/os-lab
for i in 1 2 3 4 5; do
    printf "file%d\nline%04d\n" "$i" > "file$i"
done
@uvell-G →~/os-lab (main) $ ./challenges/ch10_pool 3 file1 file2 file3 file4 file5
[parent 49345] spawned worker PID=49346 for task 'file1'
[worker 49346] START task 'file1'
[worker 49346] processed 'file1' -> lines=3 bytes=18
[worker 49346] FINISH task 'file1' (exit=0)
[parent 49345] spawned worker PID=49347 for task 'file2'
[worker 49347] START task 'file2'
[parent 49345] spawned worker PID=49348 for task 'file3'
[worker 49348] START task 'file3'
[worker 49348] processed 'file3' -> lines=3 bytes=18
[worker 49348] FINISH task 'file3' (exit=0)
[parent 49345] spawned worker PID=49349 for task 'file4'
[worker 49349] START task 'file4'
[worker 49349] processed 'file4' -> lines=3 bytes=18
[worker 49349] FINISH task 'file4' (exit=0)
[parent 49345] spawned worker PID=49350 for task 'file5'
[worker 49350] START task 'file5'
[worker 49350] processed 'file5' -> lines=3 bytes=18
[worker 49350] FINISH task 'file5' (exit=0)
[parent 49345] worker PID=49346 done (exit status=0)
[parent 49345] worker PID=49347 done (exit status=0)
[parent 49345] worker PID=49348 done (exit status=0)
[parent 49345] worker PID=49349 done (exit status=0)
[parent 49345] worker PID=49350 done (exit status=0)
[parent 49345] ALL TASKS COMPLETE (processed=5, max_concurrency=3)
@uvell-G →~/os-lab (main) $
```

Results

All programs compiled and executed successfully. Expected outputs were observed for each demonstration and challenge, including correct process behavior, exit codes, pipe communication, and concurrency limits.

Conclusion

This lab provided hands-on experience with essential operating system concepts related to process management. The exercises reinforced how UNIX systems manage processes and handle interprocess communication.